# Лабораторная работа №,2 по курсу дискретного анализа: сбалансированные деревья

Выполнил студент группы 08-215 МАИ *Матвеев Семен.*

## Условие

Требуется разработать программную библиотеку, реализующую указанную структуру данных, и на её основе разработать программу-словарь. В словаре каждому ключу, представляющему собой регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер от 0 до $2^{64} - 1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Поддерживаются следующие операции:

- `+ word 34` — добавить слово `word` с номером `34`. При успехе вывести `OK`, если слово уже есть в словаре — `Exist`.
- `- word` — удалить слово `word`. При успехе вывести `OK`, если слова нет в словаре — `NoSuchWord`.
- `word` — найти слово в словаре. При успехе вывести `OK: 34`, где число является значением, связанным со словом; если слова нет — `NoSuchWord`.
- `! Save /path/to/file` — сохранить словарь в бинарном компактном представлении. При успехе вывести `OK`, при ошибке — строку, начинающуюся с `ERROR:`.
- `! Load /path/to/file` — загрузить словарь из файла, ранее подготовленного командой `Save`. При успехе вывести `OK`; при ошибке текущий рабочий словарь должен остаться без изменений, а программа должна вывести строку, начинающуюся с `ERROR:`.

Вариант задания:

- Структура данных: красно-чёрное дерево.
- Тип ключа: регистронезависимая строка из букв английского алфавита длиной не более 256 символов.
- Тип значения: число от 0 до $2^{64} - 1$.

## Метод решения

Для хранения словаря используется красно-чёрное дерево поиска. В каждой вершине хранится ключ, значение, цвет вершины и указатели на левого сына, правого сына и родителя. Для всех отсутствующих потомков используется общий фиктивный чёрный узел `NULL_NODE`. Такой подход упрощает балансировку, потому что у каждой вершины формально есть левый и правый потомок.

### Основная идея

1. Перед вставкой, удалением и поиском ключ приводится к нижнему регистру с помощью функции ToLowerCase. За счёт этого словарь становится регистронезависимым.

2. Поиск выполняется как обычный поиск в бинарном дереве поиска: если искомый ключ меньше ключа текущей вершины, переход выполняется в левое поддерево, иначе — в правое.

3. При вставке новая вершина сначала добавляется как красная вершина. Затем вызывается функция `InsertFix`, которая восстанавливает свойства красно-чёрного дерева с помощью перекрасок и левых/правых поворотов.

4. При удалении отдельно обрабатываются случаи нуля, одного и двух детей. Если при удалении нарушается чёрная высота дерева (произошло удаление черной вершины), то вызывается функция RemoveFix. Она рассматривает цвет брата удаляемой позиции и цвета его детей, выполняя необходимые перекраски и повороты.

5. Сохранение выполняется в бинарный буфер. Для каждой вершины записывается тип узла, цвет, длина ключа, байты ключа и числовое значение.

6. Загрузка строит новое дерево из файла во временную переменную и заменяет текущее дерево только после успешного чтения всего файла.

### Немного про сложности операции

Красно-чёрное дерево является сбалансированным, потому имеет высоту $O(\log n)$. Из-за этого операции поиска, вставки и удаления выполняются за $O(\log n)$ в худшем случае. Операции `Save` и `Load` обходят все вершины дерева, поэтому их трудоёмкость равна $O(n)$, где $n$ — количество слов в словаре.

## Описание программы

**Шаблонный класс TVector<T>** — динамический массив с ручным управлением памятью. Поддерживает копирование, перемещение, доступ по индексу, изменение размера, добавление элемента в конец, очистку, итерацию через `begin`/`end`, а также получение указателя на внутренний массив через `data`. Для специализации `TVector<char>` добавлен метод `c_str`, который создаёт временную C-строку для работы с файловыми потоками.

**Тип данных TString** — псевдоним для `TVector<char>`. Он используется вместо `std::string`, чтобы программа не зависела от контейнеров STL. Для `TString` реализованы лексикографическое сравнение, сравнение с C-строкой, ввод непробельного слова из потока и вывод всех символов подряд. В операторе ввода используется флаг `was_read`: если после пропуска пробелов не был считан ни один символ, поток переводится в состояние ошибки, и вызывающий код может корректно обработать некорректную команду.

**Структура Node** — вершина красно-чёрного дерева. Содержит указатели `left`, `right`, `parent`, цвет `color`, ключ `key` типа `TString` и значение `value` типа `unsigned long long`.

**Класс RedBlackTree** — основная структура данных словаря. В приватной части класса находятся корень дерева, фиктивный узел `NULL_NODE`, функции поиска позиции вставки, повороты `RotateLeft`/`RotateRight`, балансировка после вставки и удаления, рекурсивное удаление поддерева, а также функции сериализации и десериализации. Копирование и перемещение дерева запрещены, чтобы не возникало двойного освобождения памяти.

**Функции поиска, вставки и удаления** — публичные методы `SearchNode`, `InsertNode` и `RemoveNode`. Каждый метод сначала приводит ключ к нижнему регистру, затем выполняет соответствующую операцию и печатает ответ в формате, требуемом условием задачи.

**Функции сериализации** — `WriteTreeToString`, `GetNodeString`, `AppendRawBytes`, `AppendStringBytes`, `AppendNodeTypeBytes`, `AppendColorBytes`. Дерево сохраняется рекурсивно. Берем текущий узел, записываем в буффер его тип (`NullNode`, `LeafNode` или `InnerNode`). Если наш узел имеет тип `InnerNode`, мы рекурсивно идем в левое поддерево, затем в правое поддерево. После этого записываем данные текущей вершины в буффер.

**Функции десериализации** — `ReadRawBytes`, `ReadStringBytes`, `ReadNodeFromString`, `ReadTreeFromString`. При чтении проверяются границы бинарного буфера, корректность типа узла, корректность цвета, корректность длины ключа и допустимость символов ключа. Если формат файла нарушен, уже выделенные при загрузке вершины освобождаются, а текущий словарь не заменяется.

**Функция main** — считывает входной поток посимвольно до EOF. В зависимости от первого символа строки выбирается операция добавления, удаления, сохранения/загрузки или поиска. Перед выполнением операции проверяются формат команды, корректность ключа и отсутствие лишних символов в строке.

**Помощь в обработке ввода** - функции `IsCommandEndValidAndConsumeIt` и `HandleInvalidInput`. Первая проверяет, что после корректной команды нет лишних символов до конца строки. Вторая очищает состояние потока, выводит `ERROR: Invalid command` и пропускает остаток некорректной строки.

## Дневник отладки

1. Реализовал базовый каркас словаря на красно-чёрном дереве: структуру вершины, корень, фиктивный чёрный узел `NULL_NODE`, поиск и первичную вставку без полной обработки всех пограничных случаев.

2. Реализовал балансировку после вставки. Основные ошибки возникали в поворотах: после `RotateLeft` и `RotateRight` нужно было аккуратно обновлять родителя поворачиваемой вершины, родителя нового корня поддерева и ссылку из вышестоящего узла. Отдельно пришлось обработать случай, когда поворот меняет корень всего дерева.

3. Реализовал удаление вершины. Тонким (потенциально опасным) местом программы оказался случай удаления чёрной вершины без детей: для корректной работы `RemoveFix` фиктивному узлу `NULL_NODE` временно задаётся реальный родитель удалённой позиции,

после чего родитель снова возвращается в `NULL_NODE`. Без этого балансировка не могла определить брата удалённой позиции.

4. Удаление вершины с одним ребенком пробелм не вызвало. А при удалении вершины с двумя детьми было решено не переставлять вершины физически, а скопировать в удаляемую вершину ключ и значение минимальной вершины правого поддерева, после чего удалить эту минимальную вершину. Цвет исходной вершины при этом не копируется, потому что структурно из дерева удаляется именно найденная минимальная вершина.

5. Реализовал каркас обработки пользовательского ввода (команды `+`, `-`, `! Save`, `! Load` и поиск по слову). В процессе появилась отдельная задача корректно отличать ошибку формата команды от ситуации, когда слова просто нет в словаре. Для этого была добавлена проверка конца команды и единая функция обработки некорректного ввода.

6. Реализовал сохранение дерева в бинарный файл. Идею сериализации подсказал Никита Константинович на лекции. Итоговый алгоритм рекурентно запускается из корня, сохраняет значения всех исходных полей вершины, кроме указателей. Алгоритм сериализации также использует `enum class NodeType` (пустой узел, лист и внутренняя вершина). За счёт этого загрузка может восстановить не только пары ключ-значение, но и исходную структуру дерева с цветами.

7. Реализовал загрузку дерева из файла. Важное исправление: загруженное дерево сначала строится отдельно в `loaded_root`; текущее дерево удаляется и заменяется только после того, как файл полностью прочитан, формат проверен, а буффер без ошибок прочитан до конца (смещение `offset` совпало с размером буфера).

8. На последнем этапе программа была переведена с `std::string` на собственный тип `TString`. Для этого пришлось добавить в `TVector<char>` несколько внутренних методов, сравнение строк, ввод слова из потока, поле `cstr_buffer_` для взаимодействия с `std::ifstream` и `std::ofstream` через метод `c_str`.

## Тест производительности

Для оценки эффективности реализованной структуры данных был проведён эксперимент на случайных ключах из букв английского алфавита. Для каждого размера входа генерировалась последовательность команд добавления, после чего выполнялась последовательность успешных поисковых запросов по уже добавленным ключам. Вывод программы перенаправлялся в `/dev/null`, чтобы время измерения в меньшей степени зависело от консольного вывода. Оценка производительности и, соответственно, генерация тестовых данных, была делегирована языковой модели.

Измерялись следующие операции:

- `Insert` — добавление случайных ключей в красно-чёрное дерево;
- `Search` — поиск уже существующих ключей;
- `Save + Load` — сохранение построенного дерева в бинарный файл и последующая загрузка из него.

Результаты измерений приведены в таблице.

| **Количество ключей** | **Insert, мс** | **Search, мс** | **Save + Load, мс** |
|---|---|---|---|
| 1,000 | 2.4 | 1.1 | 0.9 |
| 5,000 | 12.8 | 6.3 | 4.8 |
| 10,000 | 27.6 | 13.9 | 9.5 |
| 50,000 | 163.0 | 86.4 | 52.7 |
| 100,000 | 361.5 | 195.2 | 110.8 |

Рост времени вставки и поиска согласуется с оценкой $O(\log n)$ на одну операцию и $O(n \log n)$ для серии из $n$ операций. Время операций `Save` и `Load` растёт примерно линейно, так как при сохранении и загрузке каждая вершина дерева обрабатывается один раз.

Отдельно были проверены функциональные сценарии:

- повторное добавление существующего ключа выводит `Exist`;
- удаление отсутствующего ключа выводит `NoSuchWord`;
- поиск после удаления выводит `NoSuchWord`;
- ключи с разным регистром считаются одинаковыми;
- команда `! Load` при некорректном файле выводит `ERROR: invalid file format` и не изменяет текущий словарь;
- команды с лишними символами в конце строки выводят `ERROR: Invalid command`.

## Недочёты

- Проверка символов ключа основана на `std::isalpha`. Это норм, ноо строгая проверка именно диапазонов `a-z` и `A-Z` была бы более явно связана с условием задачи.
- При загрузке из файла я проверяю формат бинарного представления, но не выполняю валидацию свойств полученного красно-чёрного дерева: чёрный корень, отсутствие двух красных вершин подряд и одинаковая чёрная высота на всех путях.
- Функции поиска, вставки, удаления, сериализации и десериализации частично рекурсивны. Для обычных данных это допустимо из-за логарифмической высоты красно-чёрного дерева, но повреждённый файл теоретически может описывать очень глубокую структуру и привести к большому числу рекурсивных вызовов при загрузке. Можно было бы ввести максимально допустимую глубину рекурсии, но тесты программа прошла и без этого.
- Метод `c_str` в `TVector<char>` создаёт отдельный временный буфер. Это удобно для открытия файлов, но требует аккуратной инвалидизации буфера при любом изменении строки, что загрязняет код класса `TVector`.

## Выводы

В результате выполнения лабораторной работы реализована программа-словарь на основе красно-чёрного дерева. Программа поддерживает добавление, удаление, поиск, сохранение и загрузку словаря из бинарного файла. В данном словаре ключи - последовательность букв английского алфавита длиной не более 256 символов, которая обрабатывается без учёта регистра, а значения - 64-битные беззнаковые числа.

В процессе работы были реализованы методы, производящие основные операции над красно-чёрным деревов: поиск, вставка с балансировкой, удаление с балансировкой, левые и правые повороты. Также была реализована собственная строка `TString` на базе `TVector<char>`, что позволило отказаться от использования `std::string`.

Особое внимание было уделено командам `Save` и `Load`. Сохранение выполняется в компактном бинарном формате, а загрузка построена так, чтобы при ошибке чтения или несовпадении формата текущий рабочий словарь оставался без изменений.

Теоретическая оценка операций поиска, вставки и удаления составляет $O(\log n)$, что соответствует свойствам красно-чёрного дерева. Сохранение и загрузка работают за $O(n)$, так как выполняют полный обход дерева. Проведённое тестирование подтвердило ожидаемый характер роста времени работы и корректность обработки основных пользовательских сценариев.

Таким образом, задача решена в полном объёме: реализована требуемая структура данных, построена программа-словарь, добавлена обработка ошибок ввода и реализовано бинарное сохранение/восстановление состояния словаря.